การทดลองที่ 8 : ตัวชี้ (Pointer)

วัตถุประสงค์

1. เข้าใจหลักการของตัวแปรประเภทตัวชี้ และการดำเนินการของตัวชี้
2. เข้าใจและสามารถใช้ตัวชี้เพื่อส่งผ่านข้อมูลแบบอ้างอิงได้
3. เข้าใจความสัมพันธ์ระหว่างตัวชี้กับตัวแปรแถวลำดับ

ทฤษฎี

ตัวชี้เป็นตัวแปรสำหรับบันทึกตำแหน่งของหน่วยความจำ ต่างจากการบันทึกค่าของตัวแปรประเภทอื่นที่ใช้เก็บค่า การใช้งานต้องกำหนดประเภทของตัวชี้ให้ตรงกับประเภทของตัวแปรที่ต้องการนำไปใช้ แม้ว่าจะสร้างตัวชี้สำหรับตัวแปรประเภทต่างกัน ตัวชี้จะมีขนาดเท่ากัน (ขนาดของตัวแปรขึ้นอยู่กับของสถาปัตยกรรมของระบบปฏิบัติการหรือซีพียูที่คอมไพล์โปรแกรมนั้น) การนำตัวแปรประเภทตัวชี้ใช้งานอ้างอิงถึงข้อมูลอย่างอ้อม (Indirect Reference)

รูปแบบคำสั่งการประกาศตัวแปรประเภทตัวชี้
`type * pointer_name;`

`type` คือชนิดของตัวแปรที่ตัวชี้จะอ้างอิงไปถึง โดยที่

- `int` ประกาศตัวแปรที่ตัวชี้จะอ้างอิงไปถึงเป็นประเภทจำนวนเต็ม
- `float` ประกาศตัวแปรที่ตัวชี้จะอ้างอิงไปถึงเป็นประเภททศนิยม
- `char` ประกาศตัวแปรที่ตัวชี้จะอ้างอิงไปถึงเป็นประเภทอักขระ

`*` เครื่องหมายแสดงว่าเป็นตัวแปรประเภทตัวชี้

`pointer_name` ชื่อของตัวแปรประเภทตัวชี้

เมื่อประกาศตัวแปรประเภทตัวชี้แล้ว ตัวชี้นั้นจะอ้างอิงไปถึงตำแหน่งของหน่วยความจำของตัวแปรได้โดยกำหนดตำแหน่งของตัวแประให้กับตัวชี้ สัญลักษณ์ & (Address Operator) ในภาษาซี จะให้ผลลัพธ์คือตำแหน่งที่อยู่ของตัวแปรนั้น

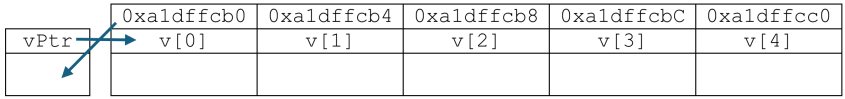
ตัวอย่าง

```
int   y = 5;      /* ประกาศตัวแปร y เป็นประเภทจำนวนเต็ม และมีค่าเริ่มต้น 5 */
int   *yPtr;      /* ประกาศตัวแปร yPtr เป็นตัวชี้ที่จะอ้างอิงไปยังตัวแปรประเภทจำนวนเต็ม */
yPtr = &y;        /* ประกาศให้ yPtr อ้างอิงไปถึงตำแหน่งของ y */
```

Variable	Address	Memory
y	0x000000583fbff7bc	5
yPtr	0x000000583fbff7b0	0x000000583fbff7bc

เมื่อตัวชี้อ้างอิงไปถึงตัวแปรใดตัวแปรหนึ่งแล้ว สามารถใช้สัญลักษณ์ * (Indirect Operator หรือ Dereference Operator) เพื่อจะอ้างถึงข้อมูล ณ ตำแหน่งที่ตัวแปรนั้นเก็บอยู่ ดังนั้นเมื่อใช้ `*yPtr` จะได้ผลลัพธ์ที่อ้างอิงถึงในกรณีนี้คือจำนวนเต็ม 5

ในภาษาซีตัวแปรประเภทตัวชี้และแถวลำดับมีความใกล้ชิดกันมาก เช่น เมื่อประกาศ `int v[5];` จะมีการจองเนื้อที่สำหรับข้อมูลประเภท Integer (ขนาด 4 ไบท์) ขึ้นมาห้าตำแหน่งติดกัน สมมติให้ตำแหน่งเริ่มต้นอยู่ที่ตำแหน่ง `0xa1dffcb0` และแต่ละตัวมีขนาดสี่ไบท์ จะมีการจัดเก็บในหน่วยความจำดังต่อไปนี้



ประเภทตัวชี้ในกรณีข้างต้นนี้ จะถูกประกาศโดยใช้คำสั่ง `int *vPtr;` และกำหนดให้ตัวชี้นี้ ชี้ไปที่ตำแหน่งเริ่มต้นของตัวแปรแถวลำดับโดยใช้วิธีใดวิธีหนึ่งดังต่อไปนี้

รูปแบบที่ 1	รูปแบบที่ 2
`vPtr = v;`	`vPtr = &v[0];`

เมื่อใช้วิธีแรกตัวชี้จะชี้ไปยังตำแหน่งเริ่มต้นคือตำแหน่งของ `v[0]` เสมอ แต่เมื่อใช้วิธีที่สอง สามารถให้ตัวชี้นั้นชี้ไปที่ตำแหน่งใด ๆ ในตัวแปรแถวลำดับได้

การใช้งานข้อมูลที่เก็บไว้ในตัวแปรแถวลำดับนั้น มีอยู่สี่วิธีตามตัวอย่างต่อไปนี้ สมมติให้มีตัวแปรประเภทตัวชี้ และแถวลำดับดังตัวอย่างข้างต้น และกำหนดให้ตัวชี้ชี้ไปยังตำแหน่งแรกของตัวแปรแถวลำดับ โดยคำสั่งใดคำสั่งหนึ่งในสองวิธีที่กล่าวไปแล้ว การเข้าถึงข้อมูลของตัวแปรแถวลำดับตำแหน่งที่ `n` สามารถทำได้โดย

1. `v[n]` เป็นการใช้งานข้อมูลแบบแถวลำดับตามแบบปกติ
2. `*(v+n)` เป็นการใช้งาน Dereference operator (*) โดยเริ่มจากตำแหน่งเริ่มต้นที่ระบุโดย `v` และเพิ่ม(หรือลด) ไปอีก `n` หน่วย สังเกตได้ว่าวิธีการนี้ใช้ตัวแปรแบบแถวลำดับแบบตัวชี้
3. `vPtr[n]` เป็นการใช้ตัวแปรประเภทตัวชี้โดยระบุตำแหน่งห่างไปจากตำแหน่งเริ่มต้น `n` หน่วย สังเกตได้ว่าวิธีการนี้ใช้ตัวชี้เสมือนว่าเป็นตัวแปรแถวลำดับ
4. `*(vPtr+n)` เป็นการใช้ข้อมูลแบบตัวชี้ตามปกติ คือใช้ Derefernce operator (*) โดยเริ่มจากตำแหน่งที่ระบุโดยตัวชี้ (vPtr) ห่างออกไปอีก `n` หน่วย

การคำนวณทางคณิตศาสตร์ของตัวชี้ เป็นอีกหัวข้อหนึ่งซึ่งพึงใช้ด้วยความระมัดระวัง การเพิ่มหรือลดค่าของตัวชี้ด้วยจำนวนเต็มใด ๆ จะเป็นการเพิ่มหรือลดค่าที่ตัวชี้นั้นเก็บอยู่คูณด้วยขนาดของวัตถุที่ตัวชี้นั้นชี้อยู่ เช่น เมื่อเป็นตัวชี้ที่ชี้ไปยังตัวเลขจำนวนเต็ม (Integer) ซึ่งมีขนาด 4 ไบท์ การเพิ่มหรือลดจะเป็นการทวีคูณของขนาด (ในกรณีนี้ คือ 4 ไบท์) เช่นหากว่า `vPtr` ชี้ไปยังตำแหน่ง `0xa1dffcb0` เมื่อใช้คำสั่ง `vPtr++` แล้ว `vPtr` จะเพิ่มไปอีกหนึ่งหน่วย (ในกรณีนี้ คือ 4 ไบท์) ตำแหน่งใหม่ที่ `vPtr` ชี้ไปจึงเป็นตำแหน่งที่ `0xa1dffcb0` + (1 หน่วย * 4 ไบท์) คือตำแหน่ง `0xa1dffcb4` ผลลัพธ์ของการคำนวณทางคณิตศาสตร์ก็เช่นเดียวกัน นั่นก็คือจะได้เป็นหน่วยของวัตถุ

ในกรณีที่ตัวชี้นั้นชี้ไปยังข้อมูลประเภทโครงสร้าง การใช้ข้อมูลในโครงสร้างนั้นทำได้โดยใช้เครื่องหมาย -> (Structure pointer operator) คือใช้รูปแบบ StructurePointer->StructureMember

การทดลองที่ 8.1 พิจารณาโปรแกรมต่อไปนี้

```
/* Lab8p1.c */
#include <stdio.h>

int main(void)
{
    int num1 = 113, num2;
    float price1 = 4.85;
    char hint1 = 'J', hint2;

    int *pt_num;
    float *pt_price;
    char *pt_hint;

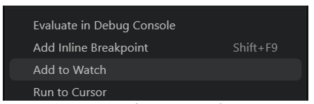
    pt_num = &num1;
    pt_price = &price1;
    pt_hint = &hint1;

    num2 = *pt_num;
    hint2 = *pt_hint;

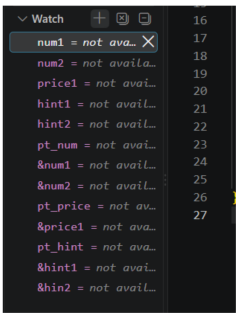
    printf("Variable num1   = %d \n", num2);
    printf("Variable price1 = %f \n", *pt_price);
    printf("Variable hint2 = %c \n", hint2);

    return 0;
} //main
```

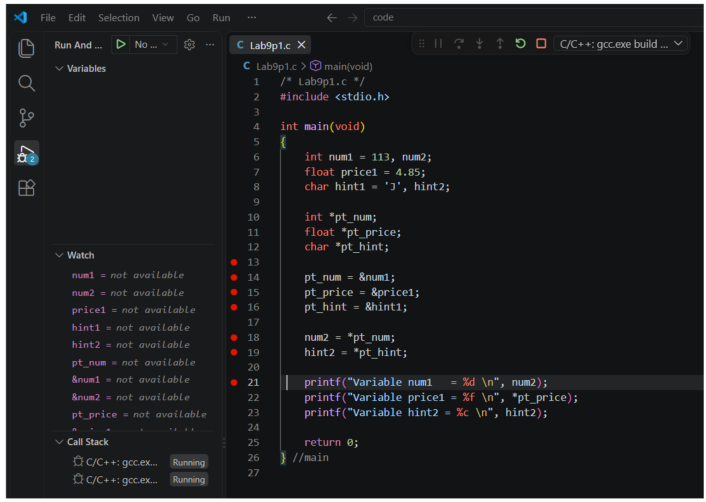
1. พิมพ์โปรแกรม Lab9p1.c
2. ฝึกการใช้งาน Add to Watch และ Debug โปรแกรม ด้วย Visual Studio Code
   a. คลิกขวาที่ตัวแปรที่ต้องการตรวจสอบค่า เลือก Add to Watch



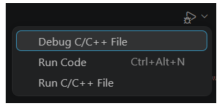
   b. เพิ่มตัวแปรตามที่ต้องการเพื่อตรวจสอบค่า (กด + แล้วพิมพ์ตัวแปร)



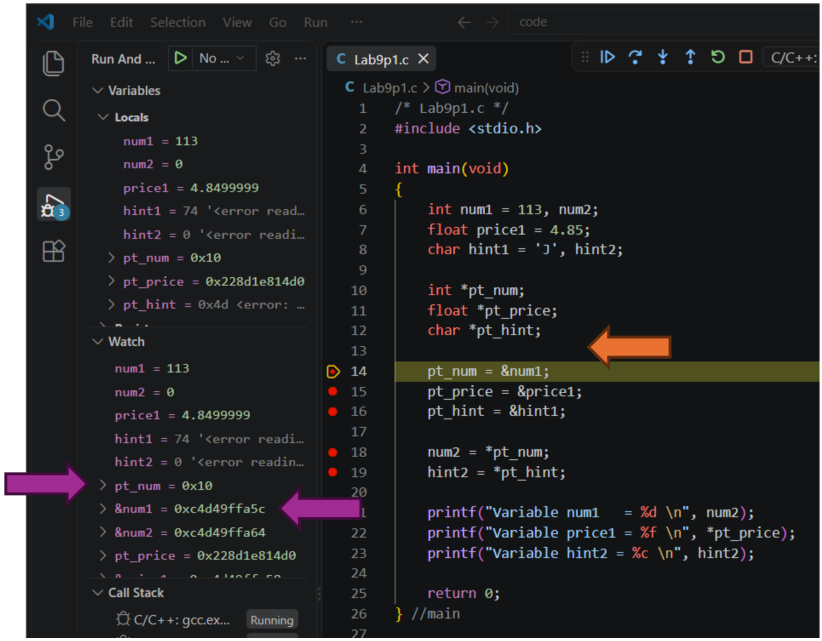
c. ใส่ Break point โดยคลิกที่หน้าเลขบรรทัดที่ต้องการเบรกให้ขึ้นจุดสีแดง



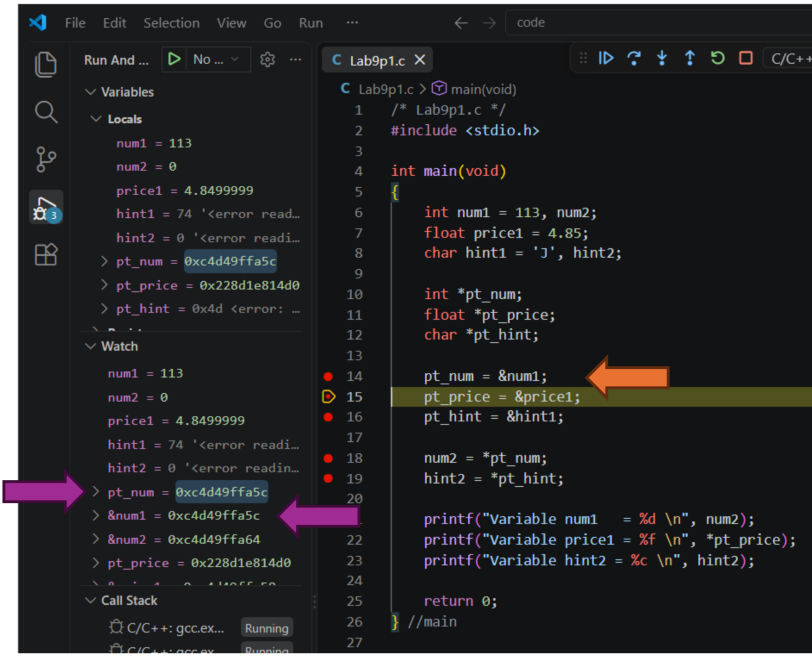
d. เลือก Debug



e. โปรแกรมจะ Run ไปหยุดบรรทัดที่ใส่ Break point ไว้ พร้อมแสดงค่าตัวแปรในหน้าต่าง Watch



f. เลือก ▶ เพื่อให้โปรแกรม Run ส่วนถัดไป (จะ Run ไปหยุดบรรทัดที่ใส่ Break point ถัดไป)



g. สังเกตการณ์เปลี่ยนแปลงข้อมูลของตัวแปรต่าง ๆ

3. บันทึกค่าต่าง ๆ ลงตาราง หากข้อมูลนั้นไม่สามารถบันทึกได้ ให้ใส่คำว่า N/A (Not Applicable)

ตัวแปร (variable)	ตำแหน่งที่อยู่ (&Variable)	ค่าที่อยู่ (value)	ค่าที่ถูกอ้างอิง (dereference value)
num1			
num2			
price1			
hint1			
hint2			
pt_num			
pt_price			
pt_hint			

การทดลองที่ 8.2 พิจารณาโปรแกรมต่อไปนี้

```
/* Lab8p2.c */
#include <stdio.h>
int main(void)
{
    char letter = 'D';
    int  num    = 19 ;
    float point = 26.09;
    char *pt_letter;
    int *pt_num;
    float *pt_point;

    pt_letter = &letter;
    pt_num = &num;
    pt_point = &point;

    printf("Address of letter = %p \n", pt_letter);
    printf("Address of num    = %p \n", pt_num);
    printf("Address of point  = %p \n", pt_point);

    return 0;
}
```

ทำตามขั้นตอนที่ให้ไว้ในการทดลองที่ 8.1 และบันทึกข้อมูลในตาราง เพื่อความเข้าใจในเนื้อหาวิชา ให้นักศึกษาพิจารณาการทำงานของโปรแกรม และคาดเดาผลลัพธ์ในแต่ละบรรทัดแต่ละขั้นตอนเสียก่อน แล้วตรวจสอบกับค่าที่ได้จากคอมไพล์เลอร์ หากไม่ตรงกันและ/หรือ ไม่เข้าใจสาเหตุที่ไม่ตรงกัน ให้นักศึกษาสอบถามกับผู้ควบคุมวิชา

ตัวแปร (variable)	ตำแหน่งที่อยู่ (&Variable)	ค่าที่อยู่ (value)	ค่าที่ถูกอ้างอิง (dereference value)
letter			
num			
point			
pt_letter			
pt_num			
pt_point			

การทดลองที่ 8.3 พิจารณาโปรแกรมต่อไปนี้

```
/* Lab8p3.c */
#include <stdio.h>
void main(void)
{
   int v[5] = { 10, 20, 30, 40, 50} /* int v[] = { 10, 20, 30, 40, 50} */
   int *v1Ptr, *v2Ptr;

   v1Ptr = &v[1];
   v2Ptr = &v[2];

   printf("The address of v1Ptr in Hex is %p and in DEC is %u\n",v1Ptr,
v1Ptr);
   printf("The address of v2Ptr in Hex is %p and in DEC is %u\n",v2Ptr,
v2Ptr);

   printf("The result of v2Ptr-v1Ptr is %d\n", v2Ptr-v1Ptr);
   printf("Size of integer is %d\n", sizeof(int));
}
```

1. ตำแหน่งของ v1Ptr คือ _____ (ฐานสิบหก) และ _____ (ฐานสิบ)
2. ตำแหน่งของ v2Ptr คือ_____ (ฐานสิบหก) และ _____ (ฐานสิบ)
3. ผลลัพธ์ของ v2Ptr – v1Ptr คือ _____
4. ขนาดของตัวแปรประเภท integer มีขนาด _____ ไบท์

การทดลองที่ 8.4 พิจารณาโปรแกรมต่อไปนี้

```
/* Lab8p4.c */
int main(void)
{
    int i, offset, b[] = {10, 20, 30, 40);
    int *bPtr = b;

    printf("Array b printed with: \n","Array Subscript notation \n");
    for (i=0; i<=3; i++)
        printf("b[%d] = %d \n",i,b[i]);

    printf("Pointer/offset notation where\nthepointer is the array name\n");
    for (offset=0; offset<=3; offset++)
        printf("*(b+%d) = %d\n",offset, *(b+offffset));

    printf("Pointer subscript notation\n");
    for (i=0; i<=3; i++)
        printf("bPtr[%d] = %d\n", i, bPtr[i]);

    printf("Pointer/offset notation\n");
    for (offset=0; offset<=3; offset++)
        printf("*(bPter+ %d) = %d\n", offset, *(bPtr+offset));

    return 0;
}
```

บันทึกผลลัพธ์จากโปรแกรมข้างต้นลงในช่องว่างนี้

การทดลองที่ 8.5 พิจารณาโปรแกรมต่อไปนี้

```
/* Lab8p5.c */
#include <stdio.h>
struct card {
       char *face;  /* 2, 3, 4, ... , J, Q, K, A */
       char *suit;  /* Spade, Heart, Diamond, Club */
};

int main(void)
{
    struct card aCard;
    struct card *cardPtr;

    aCard.face = "Ace";
    aCard.suit = "Spade";

    cardPtr = &aCard;
    printf("%s%s%s\n%s%s%s\n%s%s%s\n",
              aCard.face, " of ", aCard.suit,
              cardPtr->face, " of ", cardPtr->suit,
              (*cardPtr).face, " of ", (*cardPtr).suit);

    return 0;
}
```

บันทึกผลลัพธ์จากโปรแกรมข้างต้นลงในช่องว่างนี้

แบบฝึกหัด

1. เขียนโปรแกรม (Lab8pt_1.c) รับทศนิยม 10 ค่า และค้นหาค่าจากค่าที่ป้อนมา
    a. โดยเริ่มรับค่าจำนวนตัวเลขทศนิยม 10 จำนวน เก็บในตัวแปรแถวลำดับ
    b. ให้โปรแกรมรอรับค่าที่ต้องการค้นหามาจากผู้ใช้งาน
        i. หากพบจำนวนนั้น ให้แสดงตำแหน่ง (1-10) และค่านั้น ออกที่จอภาพ
        ii. หากไม่พบให้แสดงคำว่า “Cannot find the matched data”
    c. ให้ถามว่าผู้ใช้ต้องการค้นหาอีกหรือไม่ โดยรับอักขระ Y หรือ N
        i. ให้โปรแกรมทำงานจนกระทั่งผู้ใช้กด N

    ข้อกำหนด อนุญาตให้ใช้ตัวแปรแถวลำดับสำหรับเก็บข้อมูลทั้งสิบจำนวน จากนั้นให้ใช้ตัวชี้ชี้ไปที่ตำแหน่งเริ่มต้นของตัวแปรแถวลำดับ แล้วให้ใช้ได้เฉพาพตัวชี้เท่านั้น
2. เขียนโปรแกรม (Lab8pt_2.c) บันทึกข้อมูลนักศึกษา และเกรด และค้นหาข้อมูล
    a. ให้สร้างตัวแปรประเภทโครงสร้าง โดยให้รับข้อมูลเกี่ยวกับนักศึกษาจำนวน 5 คน โดยให้บันทึกชื่อและนามสกุล รหัสประจำตัวนักศึกษา หมายเลขโทรศัพท์ที่ให้ติดต่อ เกรดของวิชาการเขียนโปรแกรม (Programming) ฟิสิกซ์ (Physics) แคลคูลัส (Calculus) พื้นฐานคอมพิวเตอร์ (Computer Basics) โดยรับเป็นเกรด A, B, C, D, หรือ F
    b. โปรแกรมค้นหาสามารถเลือกค้นหาข้อทูลจากนักศึกษา โดยโปรแกรมจะรอรับรหัสประจำตัวนักศึกษา หากพบ ให้แสดงข้อมูล ชื่อและนามสกุล รหัสประจำตัว หมายเลขโทรศัพท์ เกรดรายวิชา และคะแนนเฉลี่ย (GPA) ของนักศึกษานั้นหากไม่พบ ให้แสดงคำว่า “ Search not found ”
    c. ให้โปรแกรมทำงานจนกว่าผู้ใช้จะป้อนรหัสนักศึกษาเป็น 0 จึงออกจากโปรแกรม

    ข้อกำหนด ให้เก็บข้อมูลของนักศึกษาเป็นตัวแปรแถวลำดับ ที่เป็นโครงสร้าง จากนั้นให้ใช้ตัวชี้ชี้ไปที่ตำแหน่งเริ่มต้นของตัวแปรแถวลำดับที่เป็นโครงสร้างนั้น แล้วให้ใช้ได้เฉพาพตัวชี้เท่านั้น
3. เขียนโปรแกรม (Lab8pt_3.c) โดยพัฒนาต่อจาก โปรแกรมข้อ 2 (Lab8pt_2.c)
    a. ให้เพิ่มสมาชิก ข้อมูลที่อยู่ และข้อมูลหมายเลขติดต่อ ในโครงสร้างเดิม เป็นประเภทโครงสร้าง
        i. โดยโครงสร้างข้อมูลที่อยู่ มีสมาชิกโครงสร้างเป็น บ้านเลขที่ (Num) ถนน (Road) ตำบล (Subdistrict) อำเภอ (District) จังหวัด (Province) และรหัสไปรษณีย์ (ZIPcode)
        ii. และโครงสร้างข้อมูลหมายเลขติดต่อ มีสมาชิกของโครงสร้างเป็น หมายเลขโทรศัพท์ปกติ (LandlineNumber) หมายเลขโทรศัพท์เคลื่อนที่ (MobileNumber) และหมายเลขโทรสาร (FaxNumber)

    ข้อกำหนด ข้อมูลของนักศึกษาแต่ละคนจะเป็นโครงสร้างซ้อนโครงสร้างอยู่ ทั้งนี้การใช้ข้อมูลตำแหน่งต่าง ๆ ให้ใช้เฉพาะตัวชี้เท่านั้น เหมือนข้อกำหนดเดิม